ShortP2P Protocol

Язык: Russian / Русский

Author: Igor Evdokimov

Repository: https://github.com/IskraMessenger/ShortP2P

Created: 2026-11-11

Modified: 2026-11-11

Related: docs/encryption, docs/messages, protocol.md

Содержание

Титульная страница

Содержание

Общие схемы

Описание протокола

Общие схемы

Уровни протокола

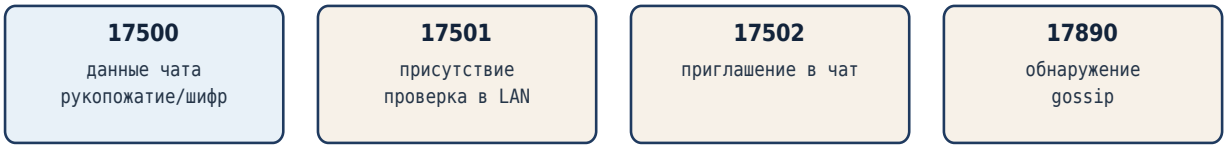


Логический стек (не уровни ОС). Подробности: docs/encryption, docs/messages.

Типичный обмен в прямом чате



UDP-порты по умолчанию



Bluetooth и mesh используют те же кадры; для Messenger Server — HTTPS.

Описание протокола

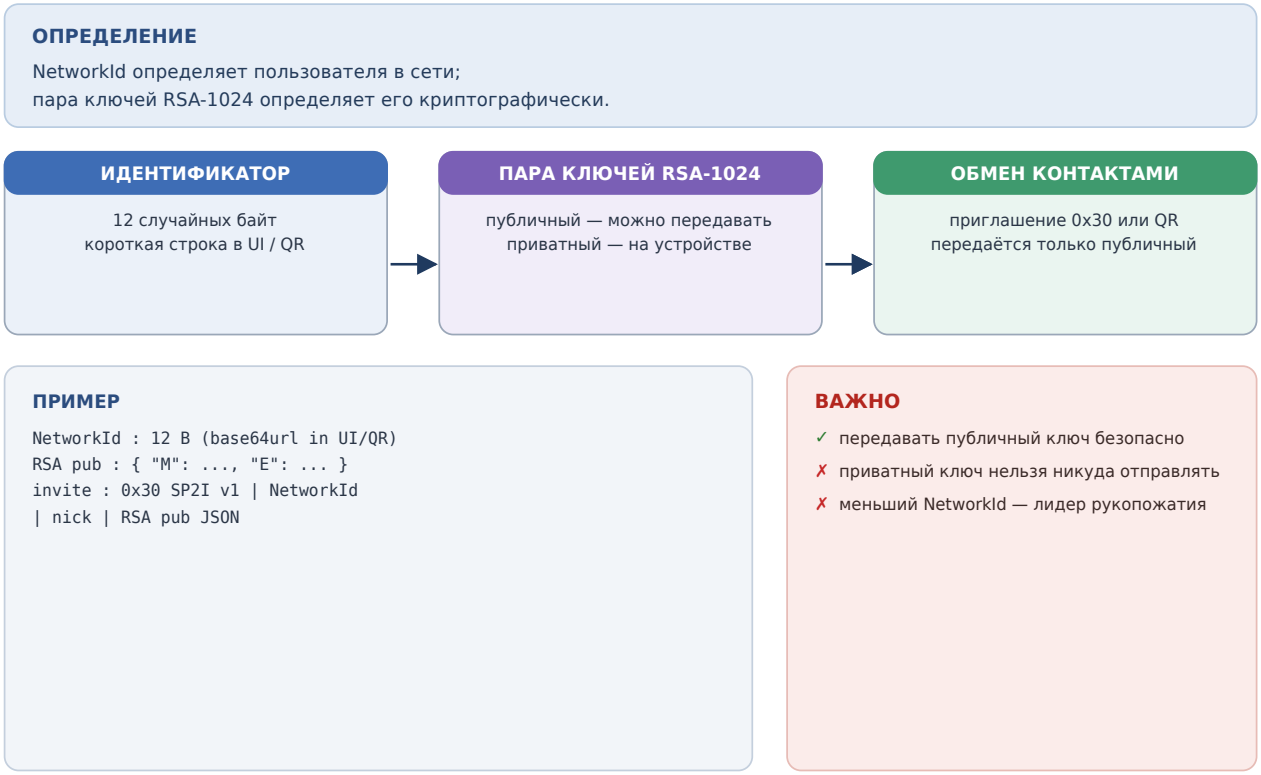
ShortP2P — стек обмена сообщениями между устройствами напрямую. Простыми словами: два устройства (пира) находят друг друга, договариваются о секретных ключах и дальше обмениваются сообщениями чата, которые никто посередине прочитать не может. Передача может идти напрямую (UDP, Bluetooth, радио mesh/LoRa) или через необязательный Messenger Server, который пересылает уже зашифрованные данные.

ОСНОВНЫЕ ПОНЯТИЯ (знание криптографии не требуется)

- Пир (peer): второе устройство или человек, с которым вы общаетесь.
- NetworkId: 12 случайных байт — идентификатор пользователя, похож на сетевой адрес. В интерфейсе и QR-кодах показывается короткой строкой base64url (~16 символов).
- RSA-1024: асимметричный алгоритм — пара ключей. Публичный ключ открыто передаётся (в приглашениях и QR-кодах); приватный никогда не покидает устройство. То, что зашифровано публичным ключом, может расшифровать только владелец приватного. В ShortP2P он используется один раз за сессию, чтобы передать пиру секреты сессии.
- Ключи сессии: две случайные строки байтов, созданные на один сеанс чата: AES-ключ (16 байт) и MAC-ключ (32 байта). После рукопожатия обе стороны держат одни и те же строки.
- AES-128-CBC: симметричный шифр — один и тот же ключ шифрует и расшифровывает. Режим CBC связывает блоки так, что одинаковые куски текста выглядят по-разному; дополнение PKCS7 выравнивает длину до кратной 16 байтам.
- IV (вектор инициализации): 16 свежих случайных байт на каждый пакет, чтобы одинаковый текст при каждом отправлении давал разный шифротекст.
- Метка HMAC-SHA256: контрольная сумма с ключом (используются 16 байт), считается по IV + шифротексту. Получатель пересчитывает и сравнивает за постоянное время; несовпадение означает, что пакет подменён или повреждён, и он отбрасывается.
- ChatWire: собственный бинарный формат ShortP2P для содержимого чата. Именно он шифруется и отправляется; получатель сначала расшифровывает, потом разбирает.
- S2P1: 4-байтовая «магическая подпись» (символы ASCII "S2P1") в начале каждого кадра ChatWire. Работает как заголовок файла — первое, что проверяет разборщик, чтобы узнать формат.
- Байт типа (kind): байт сразу после магии. Его значение говорит, какое сообщение идёт дальше.
- ACK / NACK: внутренние отчёты о доставке. ACK = «сообщение получено полностью»; NACK = «пришлите заново куски с такими номерами».
- TLS/HTTPS: стандартное шифрование канала между клиентом и Messenger Server (значок замка в браузерах).
- JWT: подписанный маркер, который сервер выдаёт при входе. Клиент прикладывает его к каждому запросу; сервер проверяет подпись, не спрашивая пароль заново.
- Сквозное шифрование (E2E): прочитать содержимое могут только два общающихся устройства. Messenger Server хранит и пересылает его как непрозрачную зашифрованную последовательность байт (текст Base64) и сам прочитать не может.
- Бот: программный аккаунт на сервере, с которым могут общаться клиенты; он тоже

получает только зашифрованное.
- Gossip / discovery: вспомогательный трафик, отвечающий на вопрос «кто рядом и как до него достучаться» (UDP-порт 17890).
- RELAY: кадр маршрутизации в локальной сети, который может нести уже зашифрованные данные через промежуточные узлы.

1) ИДЕНТИЧНОСТЬ

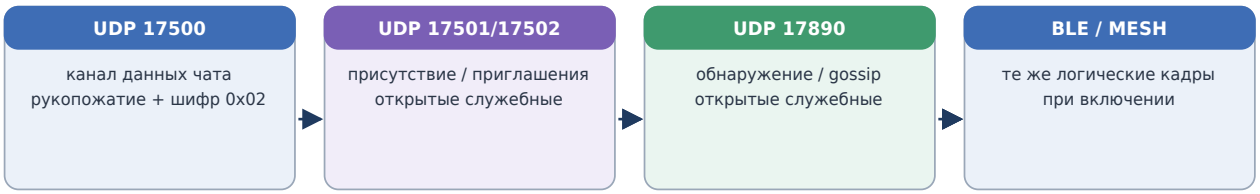


У каждого пользователя есть NetworkId (12 случайных байт) и долгосрочная пара ключей RSA-1024, созданная при регистрации. Публичный ключ передаётся открыто в приглашениях чата (0x30) или QR-кодах — это безопасно, публичный ключ и задуман для передачи.
Правило: пир с меньшим NetworkId становится «лидером» крипторукопожатия, вторая сторона ждёт.

2) ТРАНСПОРТЫ

ОПРЕДЕЛЕНИЕ

Одни и те же логические кадры ходят по нескольким транспортам;
только связь с Messenger Server всегда идёт по HTTPS.



ПРИМЕР

```
0x30 invite | 0x31 presence
0x01 handshake | 0x04 setup req
0x02 cipher | discovery: 17890
BLE / mesh: same logical frames
server: HTTPS/TLS + JWT always
```

ВАЖНО

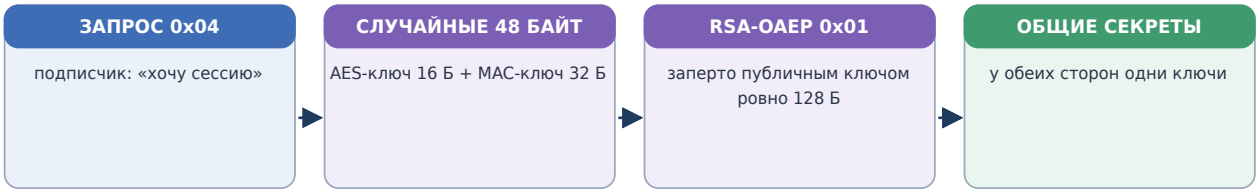
- ✓ содержимое чата — только внутри шифра 0x02
- ✗ приглашения / присутствие / gossip не шифруются
- ✗ связь с сервером — всегда HTTPS + JWT

UDP-порты по умолчанию: 17500 — канал данных чата (рукопожатие и зашифрованные пакеты), 17501 — объявления присутствия («я в сети»), 17502 — приглашения в чат, 17890 — discovery/gossip. Bluetooth и mesh/LoRa переносят те же логические кадры при включении. Связь с Messenger Server всегда идёт по HTTPS/TLS с аутентификацией JWT.

3) СЕССИЯ (прямой P2P), по шагам

ОПРЕДЕЛЕНИЕ

Лидер отправляет два случайных секрета, запертые публичным ключом пира;
дальше обе стороны шифруют одним и тем же AES-ключом.



ПРИМЕР

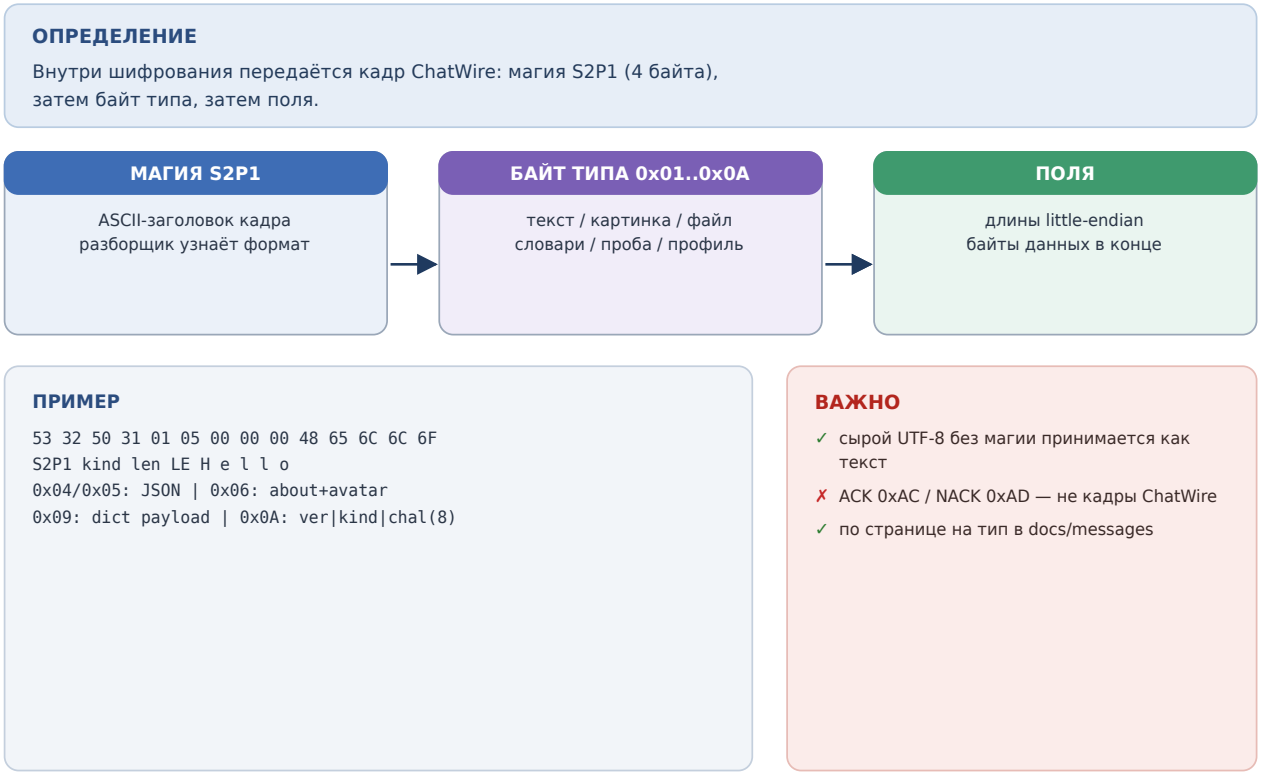
```
0x04 || NetworkId(12)
0x01 || RSA-OAEP-SHA1(128) // keys 48 B
0x02 || IV(16) || AES-CBC || tag(16)
packet <= 1024 B | plain <= 991 B
```

ВАЖНО

- ✓ сначала проверить метку HMAC, потом расшифровать
- ✗ лидер игнорирует чужой входящий 0x01
- ✗ IV — свежие случайные байты на каждый пакет

a. Подписчик может отправить кадр 0x04 со своим NetworkId: «хочу сессию».
b. Лидер генерирует два случайных секрета (AES-ключ 16 Б + MAC-ключ 32 Б) и шифрует эту 48-байтовую связку публичным ключом пира по RSA-OAEP(SHA-1). Результат — ровно 128 байт, отправляется кадром 0x01.
c. Подписчик расшифровывает своим приватным RSA-ключом; теперь у обеих сторон одни и те же два секрета.
d. Каждый следующий кадр данных — 0x02 + пакет, где пакет = IV(16) + шифротекст AES-CBC + метка HMAC(16), максимум 1024 байта. Получатель сначала проверяет метку, потом расшифровывает.

4) ПРИКЛАДНЫЕ СООБЩЕНИЯ (внутри шифрования)



Открытый текст, попадающий в шифр, — обычно кадр ChatWire: магия "S2P1", затем байт типа, затем поля (длины little-endian, если не указано иное).

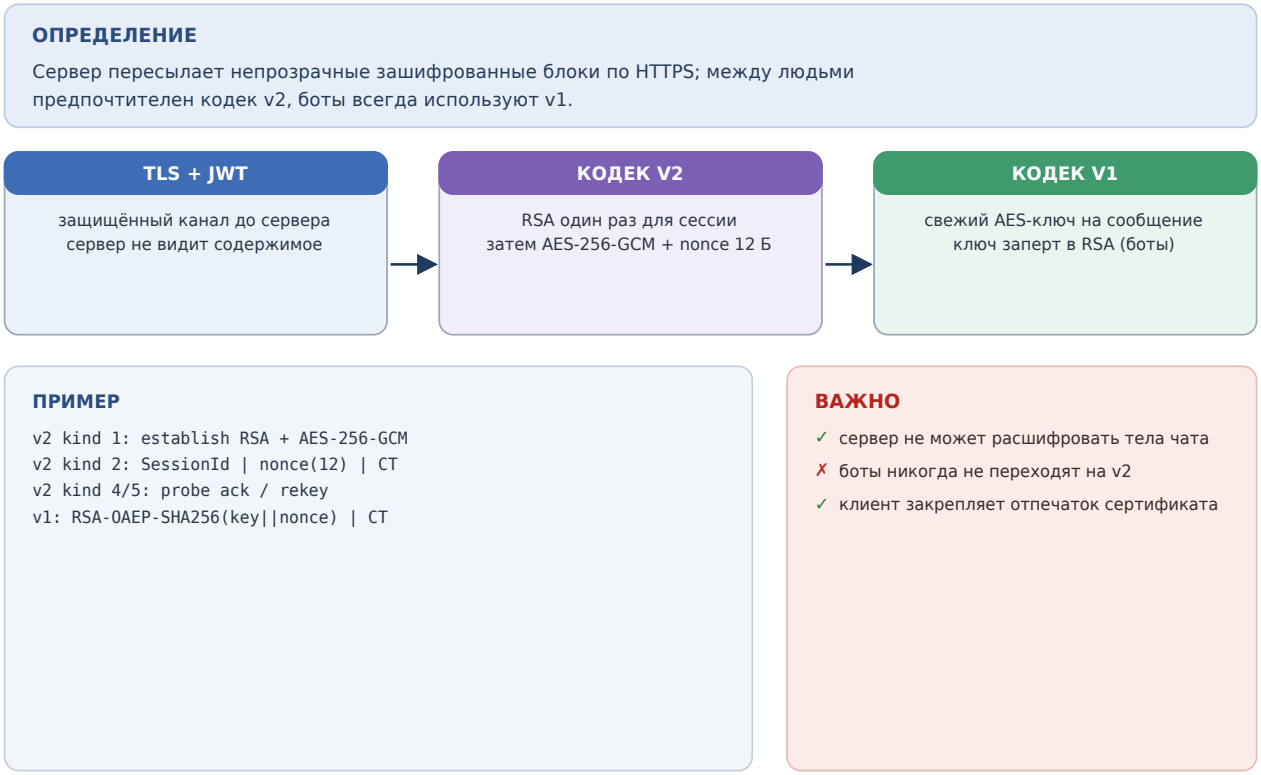
Значения байта типа, раскрыто:
- 0x01 Text — обычное текстовое сообщение чата (UTF-8), то, что набирает человек.
- 0x02 Image — картинка, передаваемая в самом чате: тип MIME (например "image/png") и байты изображения.
- 0x03 File — небольшой файл внутри чата: имя, тип MIME, байты.
- 0x04 Transfer offer — JSON-описание большого файла, предложенного для загрузки вне чата (к какому узлу/порту подключаться, одноразовый маркер, размер, срок действия).
- 0x05 Transfer control — JSON-ответ на предложение: принять, отклонить или ошибка.
- 0x06 User info — профиль отправителя («о себе» и аватар), не показывается как сообщение чата.
- 0x07 Vocabulary offer — список словарей сжатия, которые поддерживает отправитель.

- 0x08 Vocabulary accept — выбор словарей для этого чата (или "UNCOMPRESSED").
- 0x09 Compressed text — текст, закодированный выбранным словарём для экономии трафика (полезно на медленных радиоканалах).
- 0x0A Session probe — маленький проверочный сигнал («ping») через зашифрованный канал, чтобы убедиться, что обе стороны корректно расшифровывают.

Не кадры ChatWire, но тоже отправляются зашифрованно: 0xAC ACK (сообщение получено полностью, 17 байт) и 0xAD NACK (перешлите перечисленные номера кусков). Очень старые клиенты отправляют сырой UTF-8 без магии S2P1; получатель всё равно считает это текстом.

По одной странице на тип с байтовыми раскладками и незашифрованными примерами: docs/messages. Криптоконвейеры со схемами: docs/encryption.

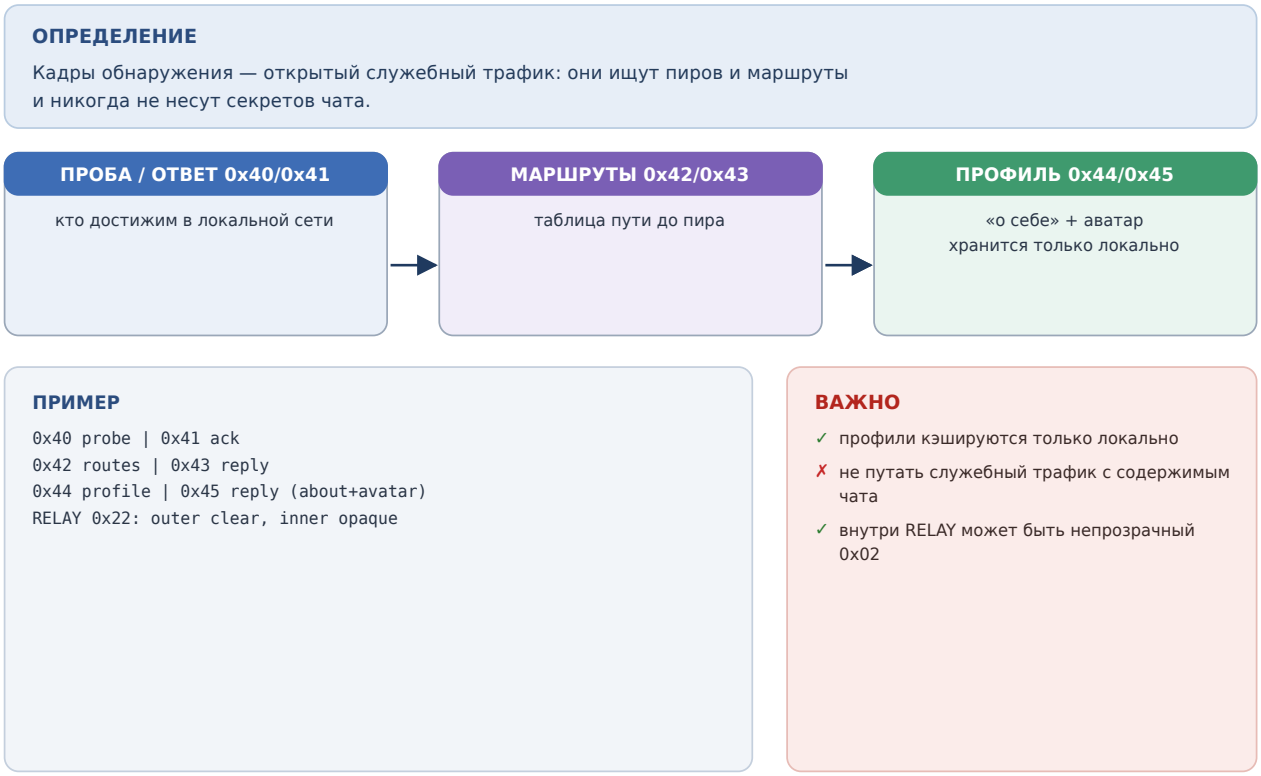
5) MESSENGER SERVER И БОТЫ



Если пиры не могут соединиться напрямую, используется Messenger Server по HTTPS/TLS+JWT. Тело чата зашифровано сквозным образом, поэтому сервер хранит только непрозрачный блок Base64.
- Кодек v2 (формат упаковки, предпочтительный между людьми): RSA используется один раз для заведения сессии; дальше каждое сообщение — AES-256-GCM (аутентифицированный шифр: шифрование и проверку целостности выполняет один алгоритм) со свежим случайным 12-байтовым nonce (одноразовым числом) на каждое сообщение.
- Кодек v1 (для ботов и как запасной путь): у каждого сообщения свой свежий ключ AES-256-GCM и nonce, а ключ+nonce оборачиваются публичным ключом получателя (RSA-OAEP-SHA256). Боты всегда используют v1.

В обоих случаях сервер не может расшифровать содержимое.

6) DISCOVERY



Кадры 0x40..0x45 на UDP 17890 (проверки/ответы, таблицы маршрутов, профили пиров) — в основном открытый служебный трафик: он помогает находить пиров, и его не надо путать с секретным содержимым чата. Кадры LAN FIND/FOUND находят адрес пира; кадр RELAY может нести уже зашифрованные данные 0x02 через промежуточные узлы.

ЧТО ЧИТАТЬ ДАЛЬШЕ

- docs/encryption — криптоконвейеры в виде векторных схем (создание сессии, форматы пакетов, серверные пути).
- docs/messages — по странице на тип сообщения с незашифрованными примерами на уровне байтов.
- protocol.md — подробное описание кадров (на русском).